

Here is a line-by-line explanation of the above block of code:

1. *defn* defines a function; *hello* is the function *name*; it takes one argument, *name*.
2. Java programmers will easily identify the *println* statement.
3. The *#* indicates that the function just defined was stored in a Clojure variable.
4. *user* is the name of the current *Namespace*, and *hello* is the function just defined.
5. The function is called with the argument as a string: 'new guy'.
6. The expected output is printed.
7. *nil* is the return value of the *println* function.

Working with Clojure source files

As you start experimenting more with Clojure, you will want to have your source code in a separate file, and then load it at the REPL. For example, here is a block of code that implements a simple number-guessing game:

```
;; Guess my number game
;; Clojure port of the Common Lisp source by Conrad Barski
;; in Land of Lisp, Chapter 2.
;; Amit Saha

;; globals
(def small 1)
(def big 100)

(defn guess-my-number []
  (bit-shift-right (+ small big) 1))

(defn smaller []
  (def big (- (guess-my-number) 1))
  (guess-my-number) )

(defn bigger []
  (def small (+ (guess-my-number) 1))
  (guess-my-number) )

(defn start-over[]
  (def small 1)
  (def big 100)
  (guess-my-number))
```

Now, load the file in the REPL using *load-file*. Let's assume you have the number 15 in your mind, and you challenge the program to guess it. Here's what a sample session would look like:

```
user=> (load-file "guessnumber.clj")
#'user/start-over
user=> (guess-my-number)
50
```

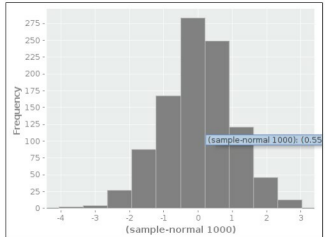


Figure 1: Incanter demo



Figure 2: Conjure Home

```
user=> (smaller)
25
user=> (smaller)
12
user=> (bigger)
18
user=> (smaller)
15
```

Editing and IDE support

Clojure is supported by most IDEs and various other editors out there. Since I personally use Emacs, I used Jeffrey Chu's Clojure mode (<https://github.com/jochu/clojure-mode>) to edit the Clojure source files.

Interaction with Java

One of the main features of Clojure, as a Lisp dialect, is that it gives you access to the vast Java toolbox. Next, we will look at some basic usage of Java classes from Clojure. This example is taken from the book *Programming Clojure*, by Stuart Halloway:

```
user=> (new java.util.Random)
#<Random java.util.Random@173ec72>
user=> (def rnd (new java.util.Random))
```